Lucrare de laborator 1
Evaluarea modelelor LLM cu acces deschis

Sarcini

- Instalați și rulați **3–5 modele LLM** local (Ollama / LM Studio) sau prin alte servicii.
- Pregătiți un set de **20 de întrebări** factuale (istorie, limba română, matematică) cu răspuns corect de referință.
- Evaluați modelele pe:
  1. **Întrebări factuale** (acuratețe/score exact/parțial)
  2. **Matematică** (aritmetică/algebră ușoară, verificare pas-cu-pas)
  3. **Eficiență** (latență, viteză de generare)
  4. **Perplexitate** (sau **pseudo-perplexitate**)
  5. Resurse/cost.
- Comparați rezultatele și faceți concluzii.

Selectați unul sau mai multe backend-uri. Completați variabilele de mai jos conform setării voastre.

- **Ollama**: `http://localhost:11434` (API `/api/generate` și `/api/chat`)
- **LM Studio**: de obicei expune un endpoint OpenAI-like: `http://localhost:1234/v1`
- **Servicii OpenAI-like**: introduceți cheie și URL.

Instrucțiuni de instalare

Ollama

1. Instalați de pe ollama.com.
2. Descărcați modele: `ollama pull llama3.1:8b`, `ollama pull qwen2.5:7b`, etc.
3. Test API: `curl http://localhost:11434/api/tags`.

Exemplu de realizare

Pregătirea spațiului de lucru

In [ ]:
```
import time, re, unicodedata, torch
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import ipywidgets as widgets
from transformers import AutoTokenizer, AutoModelForCausalLM
from ollama import Client
from ipywidgets import interact
from glob import glob
```

In [ ]:
```
client = Client(host='http://192.168.1.2:11434')
```

Vom utiliza următoarele modele (de la mare la mic):

- Qwen 3, 8b (5,2 GB)
- Llama 3.1, 8b (4,9 GB)
- Mistral, 7b (4,4 GB)
- Gemma 3, 4b (3,3 GB)
- Phi-3, 3.8b (2,2 GB)

In [ ]:
```
MODELS = (
    'gemma3:4b',
    'qwen3:8b',
    'llama3.1:8b',
    'phi3:3.8b',
    'mistral:7b'
)

def get_model_name(model):
    return model.split(':')[0]
```

Acest dicționar `GEN_OPTIONS` ajustează (tuning) comportamentul de generare a textului.

`temperature`: Controlează aleatorietatea în alegerea cuvintelor:

- Valori mici (`0.1` – `0.3`) duc la răspunsuri mai sigure, mai determinate, mai logice.
- Valori mari (`0.7` – `1.0` +) duc la răspunsuri mai variate, mai creative, dar uneori incoerente.

`top_p`: Controlează nucleul probabilităților din care se alege următorul cuvânt:

- `top_p = 0.9` înseamnă că modelul ia în considerare doar primele cuvinte a căror probabilitate cumulată ajunge la 90%.

`repeat_penalty`: Penalizează repetițiile excesive de cuvinte sau fraze.

- Valoarea mai mare decât `1` reduce probabilitatea ca modelul să repete același text.

In [ ]:
```
GEN_OPTIONS = {
    'temperature': 0.2,
    'top_p': 0.9,
    'repeat_penalty': 1.1
}
```

In [ ]:
```
SYSTEM_PROMPT =  "Ești un asistent educațional. Răspunde concis și exact \
'RĂSPUNS FINAL: <răspuns>' (fără alte cuvinte). \
Pentru probleme de matematică: arată pașii, apoi scrie pe o linie separată exact \
'RĂSPUNS FINAL: <valoare>' (fără alte cuvinte)."
```

In [ ]:
```
DATASET = [
    {'id': 1, 'cat': 'istorie', 'q': 'În ce an a avut loc Unirea Principatelor Româ
    {'id': 2, 'cat': 'istorie', 'q': 'Cine a fost domnitorul cunoscut ca „Lupul” (d
    {'id': 3, 'cat': 'istorie', 'q': 'În ce an a avut loc Marea Unire de la 1 Decem
    {'id': 4, 'cat': 'istorie', 'q': 'Capitala Republicii Moldova este?', 'ref': 'C
    {'id': 5, 'cat': 'istorie', 'q': 'Cum se numește tratatul care a încheiat Primu
    {'id': 6, 'cat': 'istorie', 'q': 'Numele domnitorului care a câștigat Bătălia d
    {'id': 7, 'cat': 'română', 'q': 'Care este pluralul corect pentru „cafea”?', 'r
    {'id': 8, 'cat': 'română', 'q': 'Indică sinonimul lui „rapid”: „iute” sau „greo
    {'id': 9, 'cat': 'română', 'q': 'Care este forma corectă: „ar trebui să vii” sa
    {'id': 10, 'cat': 'română', 'q': 'Ce parte de vorbire este cuvântul „frumos” în
    {'id': 11, 'cat': 'română', 'q': 'Care este antonimul lui „adevăr”: „minciună”
    {'id': 12, 'cat': 'română', 'q': 'Spune forma de genitiv-dativ plural pentru „e
    {'id': 13, 'cat': 'matematica', 'q': 'Calculează: 245 + 378 =', 'ref': '623', '
    {'id': 14, 'cat': 'matematica', 'q': 'Calculează: 1000 - 457 =', 'ref': '543',
    {'id': 15, 'cat': 'matematica', 'q': 'Calculează: 36 × 12 =', 'ref': '432', 're
    {'id': 16, 'cat': 'matematica', 'q': 'Calculează: 560 ÷ 8 =', 'ref': '70', 'ref
    {'id': 17, 'cat': 'matematica', 'q': 'Rezolvă pentru x: 2x + 5 = 17', 'ref': '6
    {'id': 18, 'cat': 'matematica', 'q': 'Rezolvă pentru x: 3x - 4 = 2', 'ref': '2'
    {'id': 19, 'cat': 'matematica', 'q': 'Calculează: (15 + 5) × (10 - 7) =', 'ref'
    {'id': 20, 'cat': 'matematica', 'q': 'Calculează: 7² + 5² =', 'ref': '74', 'ref
]
```

Metoda `normalize_text` elimină diacritice, și încearcă să extragă răspunsul final dintr-un text generat, stocat într-un set.

In [ ]:
```
FINAL_ANSWER_TEXT = 'raspuns final:'

def normalize_text(text, split):
    text = ''.join(c for c in unicodedata.normalize('NFD', text) if unicodedata.cat

    if split:
        possible_ans = text.rfind(FINAL_ANSWER_TEXT)

        if possible_ans != -1:
```

```
            text = text[possible_ans + len(FINAL_ANSWER_TEXT):]

    text = set(re.findall(r'\w+', text))
    return text
```

Aici avem o metodă ce calculează similaritatea Jaccard între seturi.

In [ ]:
```
def jaccard_tokens(a, b):
    if len(a) == 0 and len(b) == 0:
        return 1.0

    if len(a) == 0 or len(b) == 0:
        return 0.0

    return len(a & b) / len(a | b)
```

Comparăm răspunsul generat cu cel de referință.

In [ ]:
```
def factual_scores(model_answer, ref):
    a = normalize_text(model_answer, True)
    b = normalize_text(ref, False)

    partial = jaccard_tokens(a, b)

    return {
        'exact': int(partial > .99999999),
        'partial': partial
    }
```

Utilizăm regex pentru a extrage un număr întreg sau real din răspunsul final.

In [ ]:
```
NUMBER_PATTERN = r'[+-]?\d+(?:[.,]\d+)?'
FINAL_MATH_ANS_RE = re.compile(fr'RĂSPUNS FINAL:\s*({NUMBER_PATTERN})', re.IGNORECA

def parse_final_number(text):
    m = FINAL_MATH_ANS_RE.search(text)

    if m is None:
        candidates = re.findall(NUMBER_PATTERN, text)

        if not candidates:
            return np.nan

        val = candidates[-1]
    else:
        val = m.group(1)

    val = val.replace(',', '.')

    try:
        return float(val)
    except:
        return np.nan
```

In [ ]:
```python
MATH_ANS_RE = re.compile(r'(^|\n)\s*(pas(ul)?|\d+[.)])', re.IGNORECASE)

def math_score(model_answer, ref_num):
    steps = int(bool(MATH_ANS_RE.search(model_answer)))
    num = parse_final_number(model_answer)

    if num == np.nan:
        correct = 0
    else:
        correct = int(abs(num - ref_num) <= 1e-6)

    return {'correct': correct, 'extracted': num, 'has_steps': steps}
```

Ollama nu oferă evaluări mai avansate pe tokeni (`logprobs`), deci nu putem calcula perplexitatea intrinsecă a modelului evaluat.

În schimb, folosim un modelul de referință, GPT-2, pentru a calcula pseudo-perplexitate pentru textul generat.

GPT-2 are context limitat (1024 tokeni). Pentru texte lungi, folosim sliding window cu variabila `stride` pentru a evita depășirea contextului.

Sursă: https://huggingface.co/docs/transformers/perplexity

In [ ]:
```python
REF_LM = 'gpt2'
tok = AutoTokenizer.from_pretrained(REF_LM)
lm = AutoModelForCausalLM.from_pretrained(REF_LM)
lm.eval()

def ref_perplexity(text):
    stride = 512

    enc = tok(text, return_tensors='pt')
    input_ids = enc['input_ids']
    max_length = lm.config.n_positions

    nlls = []

    for i in range(0, input_ids.size(1), stride):
        begin_loc = max(i + stride - max_length, 0)
        end_loc = min(i + stride, input_ids.size(1))

        trg_len = end_loc - i
        input_ids_slice = input_ids[:, begin_loc:end_loc]

        target_ids = input_ids_slice.clone()
        target_ids[:, :-trg_len] = -100

        with torch.no_grad():
            out = lm(input_ids_slice, labels=target_ids)
            neg_log_likelihood = out.loss * trg_len

        nlls.append(neg_log_likelihood)
```

```
    ppl = torch.exp(torch.stack(nlls).sum() / input_ids.size(1))
    return float(ppl)
```

Normalizăm datele de performanță obținute de la Ollama.

In [ ]:
```
def perf_from_final_chunk(first_token_time, start_time, last_chunk):
    def to_ms(ns):
        return (ns or 0) / 1e6

    eval_ms = to_ms(last_chunk.eval_duration)
    gen_tokens = int(last_chunk.eval_count)

    return {
        'ttft_ms': max(0.0, (first_token_time - start_time) * 1000.0),
        'total_ms': to_ms(last_chunk.total_duration),
        'load_ms': to_ms(last_chunk.load_duration),
        'prompt_eval_ms': to_ms(last_chunk.prompt_eval_duration),
        'eval_ms': eval_ms,
        'prompt_tokens': int(last_chunk.prompt_eval_count),
        'gen_tokens': gen_tokens,
        'tps': gen_tokens / (eval_ms / 1000.0)
    }
```

Funcția ce ne permite să expediem un prompt către un model.

De asemenea, putem specifica dacă vrem ca modelul să fie tuned.

In [ ]:
```
def ask_model(model, user_prompt, tune):
    start = time.time()
    content = []
    first_token_ts = None
    last_chunk = None

    options = { 'num_ctx': 4096 }

    if tune:
        options.update(GEN_OPTIONS)

    stream = client.chat(
        model=model,
        messages=[
            {'role':'system', 'content': SYSTEM_PROMPT},
            {'role':'user', 'content': user_prompt}
        ],
        stream=True,
        options=options
    )

    for chunk in stream:
        last_chunk = chunk
        piece = chunk.message.content or ''

        if len(piece) > 0 and first_token_ts is None:
            first_token_ts = time.time()
```

```
        content.append(piece)

    full = ''.join(content)

    if first_token_ts is None:
        first_token_ts = time.time()

    perf = perf_from_final_chunk(first_token_ts, start, last_chunk)
    return full, perf
```

Interacțiune

Interactiv, putem evalua un model la alegere, pe cele 20 de întrebări.

Benchmark-ul/rezultatele se stochează într-un fișier `.json`.

In [ ]:
```
@interact(model=widgets.Dropdown(
    options=[None, *MODELS],
    description='Model:'
))
def run_benchmark(model):
    if model is None:
        return

    rows = []

    for item in DATASET:
        for tuned in (False, True):
            full, perf = ask_model(model, item['q'], tuned)
            ppl = ref_perplexity(full)

            row = {
                'model': model,
                'tuned': tuned,
                'id': item['id'],
                'cat': item['cat'],
                'q': item['q'],
                'ref': item['ref'],
                'answer': full,
                'exact': np.nan,
                'partial': np.nan,
                'math_correct': np.nan,
                'has_steps': np.nan,
                'ttft_ms': perf['ttft_ms'],
                'tps': perf['tps'],
                'gen_tokens': perf['gen_tokens'],
                'total_ms': perf['total_ms'],
                'eval_ms': perf['eval_ms'],
                'prompt_eval_ms': perf['prompt_eval_ms'],
                'load_ms': perf['load_ms'],
                'ref_ppl': ppl,
            }
```

```
            match item['cat']:
                case 'matematica':
                    ms = math_score(full, item['ref_num'])
                    row['math_correct'] = ms['correct']
                    row['has_steps'] = ms['has_steps']

                case _:
                    sc = factual_scores(full, item['ref'])
                    row['exact'] = sc['exact']
                    row['partial'] = sc['partial']

            rows.append(row)

    pd.DataFrame(rows).to_json(f'benchmarks/{get_model_name(model)}.json', orient='
```

```
interactive(children=(Dropdown(description='Model:', options=(None, 'gemma3:4b', 'qw
en3:8b', 'llama3.1:8b', 'p…
```

Îmbinăm rezultatele din fișierele `.json` într-un singur `DataFrame` pentru statistică.

In [ ]:
```
results_df = None

for file in glob('benchmarks/*.json'):
    temp = pd.read_json(file)

    if results_df is None:
        results_df = temp
        continue

    results_df = pd.concat([results_df, temp], ignore_index=True)
```

In [ ]:
```
results_df
```

Out[ ]:

	model	tuned	id	cat	q	ref	answer	exact
0	gemma3:4b	False	1	istorie	În ce an a avut loc Unirea Principatelor Român...	1859	Mica Unire a avut loc în 1859.\n\nRĂSPUNS FINA...	1.0
1	gemma3:4b	True	1	istorie	În ce an a avut loc Unirea Principatelor Român...	1859	Mica Unire a avut loc în 1859.\n\nRĂSPUNS FINA...	1.0
2	gemma3:4b	False	2	istorie	Cine a fost domnitorul cunoscut ca „Lupul” (do...	Mircea cel Bătrân	Mihai Viteazul.\n\nRĂSPUNS FINAL: Mihai Viteazul	0.0
3	gemma3:4b	True	2	istorie	Cine a fost domnitorul cunoscut ca „Lupul” (do...	Mircea cel Bătrân	Mihail Viteazul.\n\nRĂSPUNS FINAL: Mihail Vite...	0.0
4	gemma3:4b	False	3	istorie	În ce an a avut loc Marea Unire de la 1 Decemb...	1918	1867\n\nRĂSPUNS FINAL: 1867	0.0
...	...	...	...	...	...	...	...	...
195	qwen3:8b	True	18	matematica	Rezolvă pentru x: 3x - 4 = 2	2	<think>\nOkay, let's see. I need to solve the ...	NaN
196	qwen3:8b	False	19	matematica	Calculează: (15 + 5) × (10 - 7) =	60	<think>\nOkay, let's tackle this math problem....	NaN
197	qwen3:8b	True	19	matematica	Calculează: (15 + 5) × (10 - 7) =	60	<think>\nOkay, let's tackle this math problem....	NaN
198	qwen3:8b	False	20	matematica	Calculează: $7^2 + 5^2$ =	74	<think>\nOkay, let's see. The user wants me to...	NaN
199	qwen3:8b	True	20	matematica	Calculează: $7^2 + 5^2$ =	74	<think>\nOkay, let's see. The user wants me to...	NaN

200 rows × 19 columns

Agregarea benchmark-urilor

In [ ]:
```
agg = results_df.groupby(['model', 'tuned']).agg(
    factual_exact=('exact', 'mean'),
    factual_partial=('partial', 'mean'),
    math_acc=('math_correct', 'mean'),
    steps_rate=('has_steps', 'mean'),
    ttft_ms=('ttft_ms', 'mean'),
    tps=('tps', 'mean'),
    ref_ppl=('ref_ppl', 'mean'),
    gen_tokens=('gen_tokens', 'mean'),
).reset_index()

agg
```

Out[ ]:

	model	tuned	factual_exact	factual_partial	math_acc	steps_rate	ttft_ms	
0	gemma3:4b	False	0.750000	0.800000	0.750	0.375	278.191245	53.0
1	gemma3:4b	True	0.750000	0.800000	0.875	0.375	80.511582	52.8
2	llama3.1:8b	False	0.833333	0.833333	1.000	0.750	169.256759	55.4
3	llama3.1:8b	True	0.833333	0.843750	0.875	0.875	52.624249	55.0
4	mistral:7b	False	0.666667	0.689394	0.500	0.500	296.018934	58.9
5	mistral:7b	True	0.666667	0.722222	0.875	0.625	27.552032	58.6
6	phi3:3.8b	False	0.000000	0.002222	0.500	0.125	193.215632	85.3
7	phi3:3.8b	True	0.083333	0.083333	0.750	0.250	25.446188	85.6
8	qwen3:8b	False	0.750000	0.760417	1.000	0.250	430.424178	45.8
9	qwen3:8b	True	0.666667	0.666667	1.000	0.500	49.969661	45.0

Atribut	Descriere
model	Numele modelului LLM evaluat (ex. `llama3.1`, `mistral:7b`).
tuned	Starea modelului: `False` = original, `True` = ajustat/tunat.
factual_exact	Procentul întrebărilor factuale cu răspuns exact (0–1).
factual_partial	Scor mediu de similaritate parțială (Jaccard) între răspuns și referință (0–1).
math_acc	Procentul întrebărilor de matematică rezolvate corect (0–1).
steps_rate	Procentul răspunsurilor la matematică care includ pași explicativi (0–1).
ttft_ms	Time-To-First-Token în milisecunde (latența inițială).
tps	Tokens per second (viteza de generare).

Atribut	Descriere
ref_ppl	Ref-PPL (pseudo-perplexitate) calculată cu un model de referință (mai mic = mai fluent).
gen_tokens	Numărul mediu de tokeni generați per răspuns.

In [ ]:
```
sns.set_theme(style='whitegrid')

fig, axes = plt.subplots(2, 3, figsize=(13, 7))

sns.barplot(data=agg, x='model', y='factual_exact', hue='tuned', legend=False, ax=a
axes[0,0].set_title('Acurateţe factuală (exact)')

sns.barplot(data=agg, x='model', y='factual_partial', hue='tuned', legend=False, ax
axes[0,1].set_title('Acurateţe factuală (parţial)')

sns.barplot(data=agg, x='model', y='math_acc', hue='tuned', legend=False, ax=axes[0
axes[0,2].set_title('Matematică (corect)')

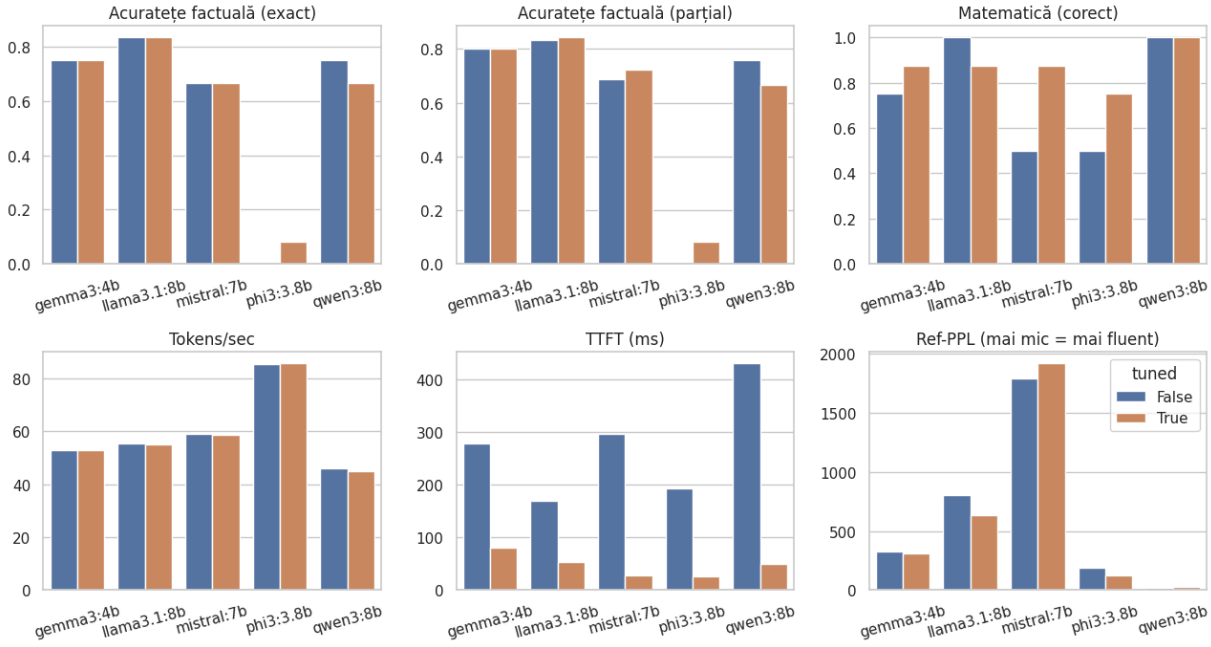
sns.barplot(data=agg, x='model', y='tps', hue='tuned', legend=False, ax=axes[1,0])
axes[1,0].set_title('Tokens/sec')

sns.barplot(data=agg, x='model', y='ttft_ms',hue='tuned', legend=False,  ax=axes[1,
axes[1,1].set_title('TTFT (ms)')

sns.barplot(data=agg, x='model', y='ref_ppl', hue='tuned', ax=axes[1,2])
axes[1,2].set_title('Ref-PPL (mai mic = mai fluent)')

for ax in axes.ravel():
    ax.set_xlabel('')
    ax.set_ylabel('')
    ax.tick_params(axis='x', rotation=15)

plt.tight_layout()
plt.show()
```



Putem exporta statisticile.

In [ ]:
```python
def save_json(df, name):
    df.to_json(name, orient='records', indent=4, index=False)

save_json(results_df, 'ollama_eval_item_level.json')
save_json(agg, 'ollama_eval_aggregate.json')
```

Concluzii

Cel mai bun la factual (exact și parțial)

Modelul `llama3.1:8b` (tuned) are cele mai bune rezultate factuale, cu:

- `factual_exact` ≈ 83%
- `factual_partial` ≈ 84%

Depășește celelalte modele atât în versiunea netunată, cât și în cea ajustată (tuned).

Cel mai bun la matematică

Tot `llama3.1:8b` (netuned) obține `math_acc` = 100%, urmat de `qwen3:8b` (atât tuned cât și netuned), care atinge de asemenea 100% la matematică, dar cu performanță factuală mai slabă.

Cel mai eficient (viteză)

După metrica `ttft_ms` (time-to-first-token) și `tps` (tokens per second):

- Cel mai rapid timp de răspuns: `mistral:7b` (tuned) ≈ 27.55 ms TTFT

- Cel mai bun throughput: `mistral:7b` (tuned) ≈ 58.65 TPS

Cel mai flexibil (Ref-PPL mai mic = mai coerent)

`qwen3:8b` (netuned) are cea mai mică `ref_ppl` ≈ 23, indicând o coerență superioară și o mai bună perplexitate a textului generat.

Compromis între calitate și viteză

`llama3.1:8b` oferă cel mai echilibrat compromis:

- factual și matematic foarte bune pe toate metricele,
- viteză bună (TTFT ≈ 52.6 ms, TPS ≈ 55),
- și coerență rezonabilă (Ref-PPL ≈ 634).